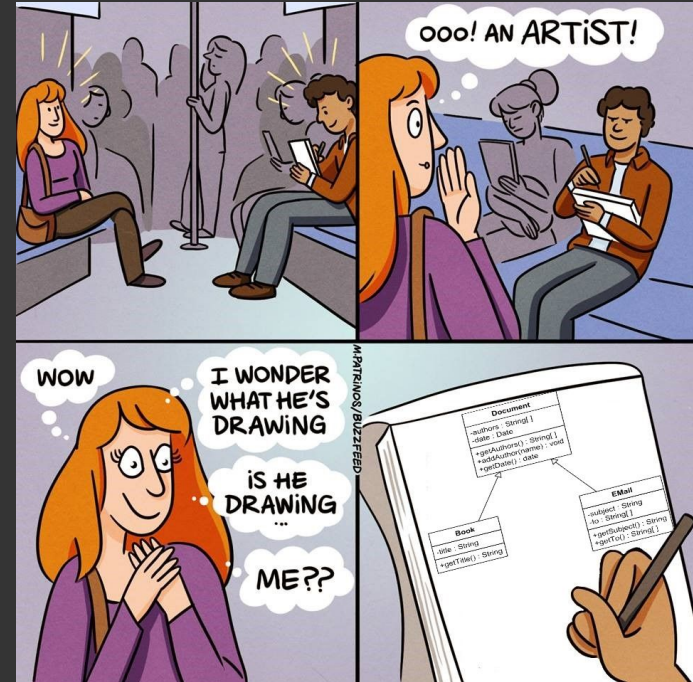


Object Oriented Analysis and Design

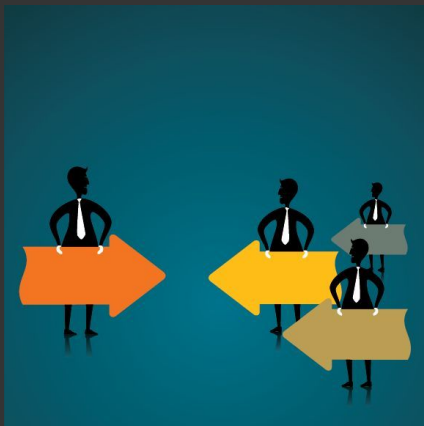
UML and Class Diagrams

Exercise 4



UML Diagrams

- Structure diagrams
- Behavior diagrams



Class diagram

- Describe the connections between classes in our system

<<Interface>>

Abstract

Class

Attributes

Behaviors

Aggregation



Composition



This is a note

Inheritance



+

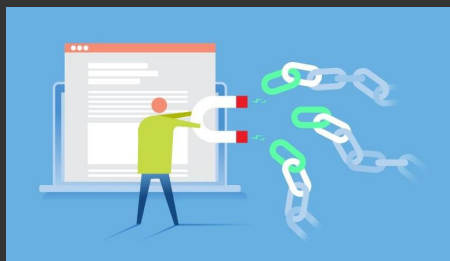
Public

-

Private

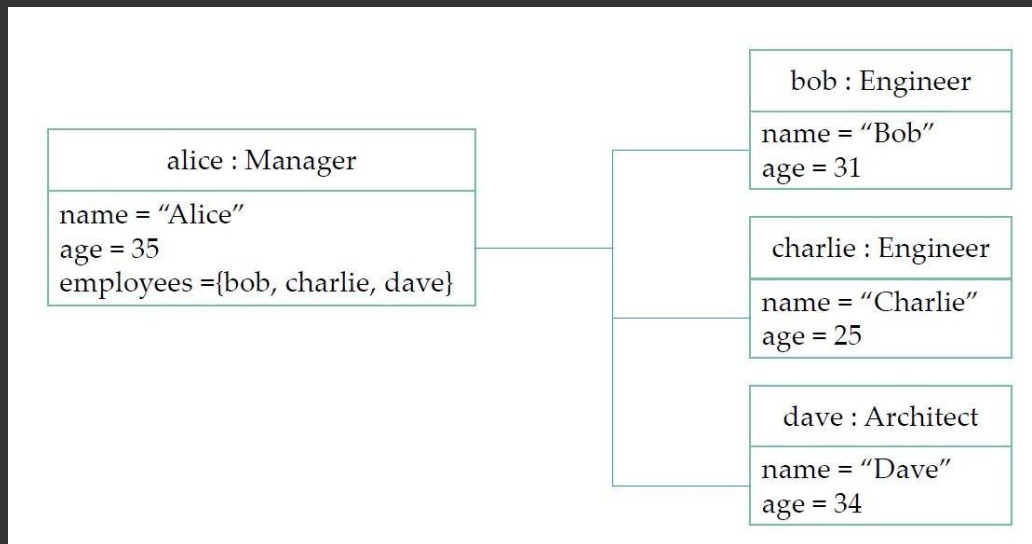
#

Protected



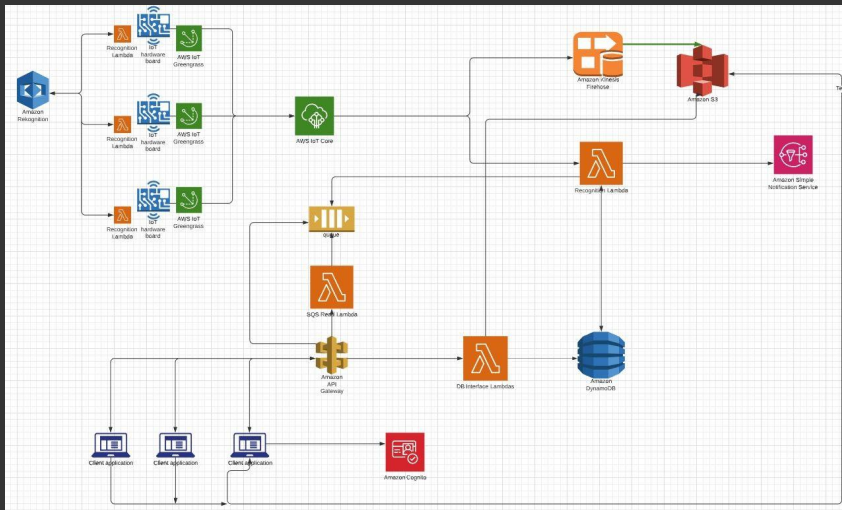
Object diagram

- Describe the connections between Objects



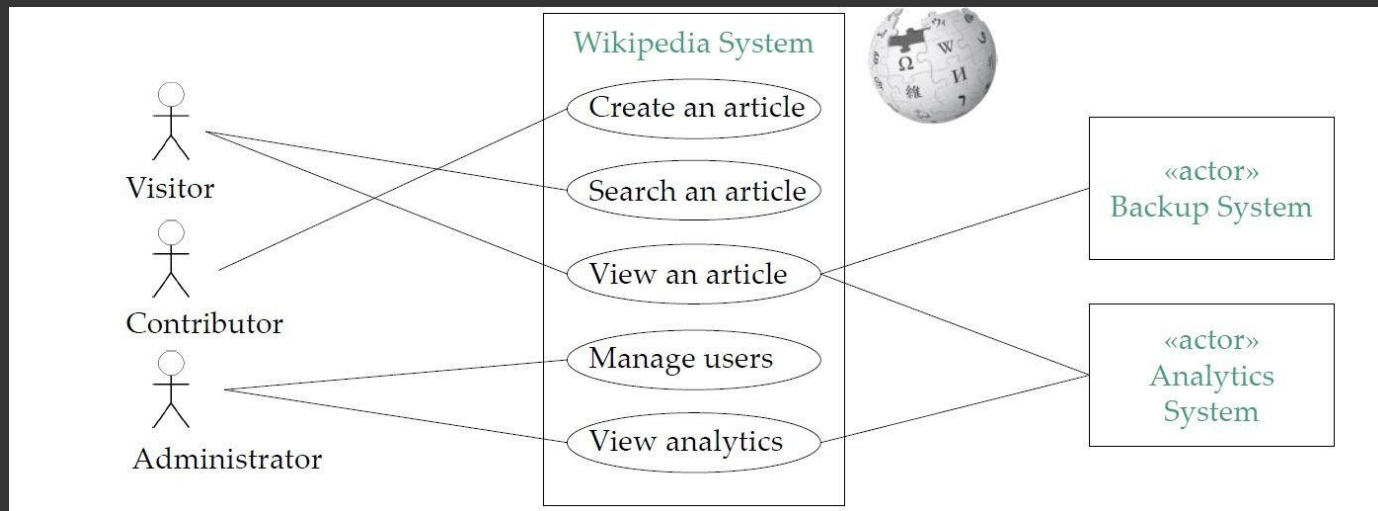
diagram

- Describe the connections between system components



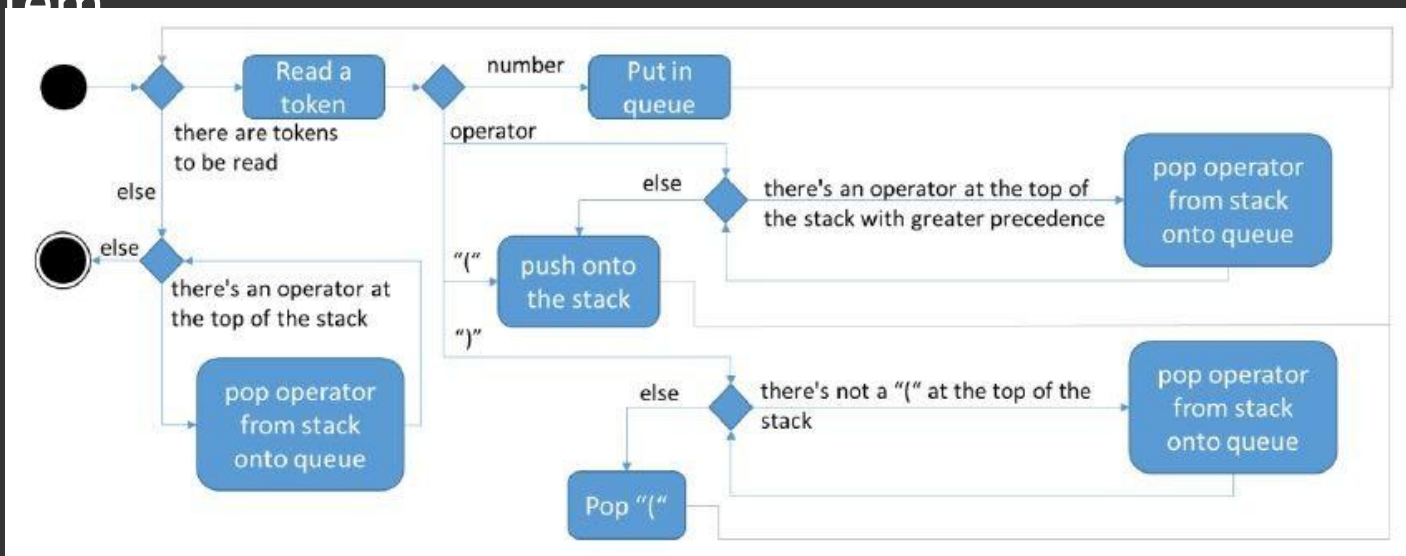
Use case diagram

- Describe the flow between actors to use specific use case



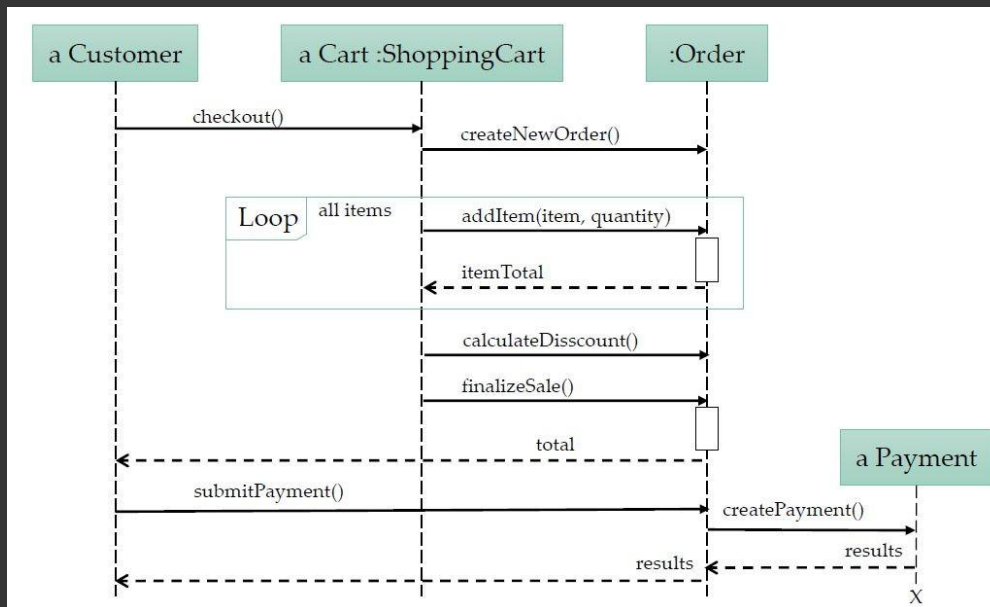
Activity diagram

Describe general flow of our system



Sequence diagram

describes specific use case flow over time



Types of Relationships

- Generalization (inheritance)
- Composition
- Aggregation
- Association
- Dependency



Types of Relationships

OOP Class Relationships Overview

Relationship	Meaning	Tightness	Lifetime
Inheritance	IS-A	Tight	Permanent
Composition	HAS-A	Tight	Part dies with whole
Aggregation	HAS-A	Loose	Part survives whole
Association	Uses-A / Knows-A	Loose	Independent
Dependency	Uses-A temporarily	Very loose	Temporary

Inheritance

- **What it is:** A class inherits from another class, taking on its properties and methods.
- **Real-life example:** A Dog is an Animal. The Dog class inherits from Animal
- **Purpose:** Reuse and extend code. Dog gets everything Animal has, and can override or add new behavior.

```
#include <iostream>
using namespace std;

class Animal {
public:
    void speak() {
        cout << "Animal sound" << endl;
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Bark!" << endl;
    }
};
```

Composition

- **What it is:** One class has another class as a field (instance variable).
- **Real-life example:** A Car has an Engine.
- **Purpose:** Build complex objects out of simpler ones. It's flexible and encourages modularity.

```
class Engine {  
public:  
    void start() {  
        cout << "Engine started" << endl;  
    }  
};  
  
class Car {  
private:  
    Engine engine; // Strong ownership  
  
public:  
    void startCar() {  
        engine.start();  
    }  
};
```

Aggregation

- **What it is:** Like composition, but the part can exist independently of the whole.
- **Real-life example:** A Team has Players, but players can exist without a team.
- **Purpose:** Represents shared ownership, not tight coupling.

```
#include <iostream>
using namespace std;

class Player {
public:
    string name;
    Player(string name) : name(name) {}
};

class Team {
private:
    Player* player; // Weak ownership (pointer)

public:
    Team(Player* p) : player(p) {}

    void showPlayer() {
        cout << "Team player: " << player->name << endl;
    }
};
```

Association

- **What it is:** A loose connection between two classes; one knows about or uses the other.
- **Real-life example:** A Teacher uses a Whiteboard, but the Whiteboard exists independently.
- **Purpose:** Enables collaboration without strong ownership.

```
#include <iostream>
using namespace std;

class Whiteboard {
public:
    void write() {
        cout << "Writing on the board" << endl;
    }
};

class Teacher {
private:
    Whiteboard* board; // Long-term association

public:
    void setWhiteboard(Whiteboard* wb) {
        board = wb;
    }

    void teach() {
        if (board) {
            board->write();
        }
    }
};
```

Dependency

- **What it is:** One class depends on another to do its job, usually passed in or used temporarily.
- **Real-life example:** A PrinterService needs a Printer to print.
- **Purpose:** Very loose, often used in method parameters (temporary collaboration).

```
class Printer {  
public:  
    void printDocument() {  
        cout << "Printing document..." << endl;  
    }  
};  
  
class ReportGenerator {  
public:  
    void generate(Printer& printer) {  
        // Temporary use of Printer  
        printer.printDocument();  
    }  
};
```

Class Diagram Examples

The Library Scenario

The library allows users to register as members. Each member can borrow up to 5 books at once, tracked by a loan.

A Book has a title, an author (can be shared), and a unique ID. Books can either be physical books or e-Books — physical books have shelf locations, e-Books have download links.

Authors can write multiple books and live independently of the library system.

Books are grouped into sections, which are managed by the library. Sections cannot exist independently from a library.

A librarian can register members and manage inventory.

The system should support the feature of future book reservation. This feature can be extended to other media later (e.g., DVDs).

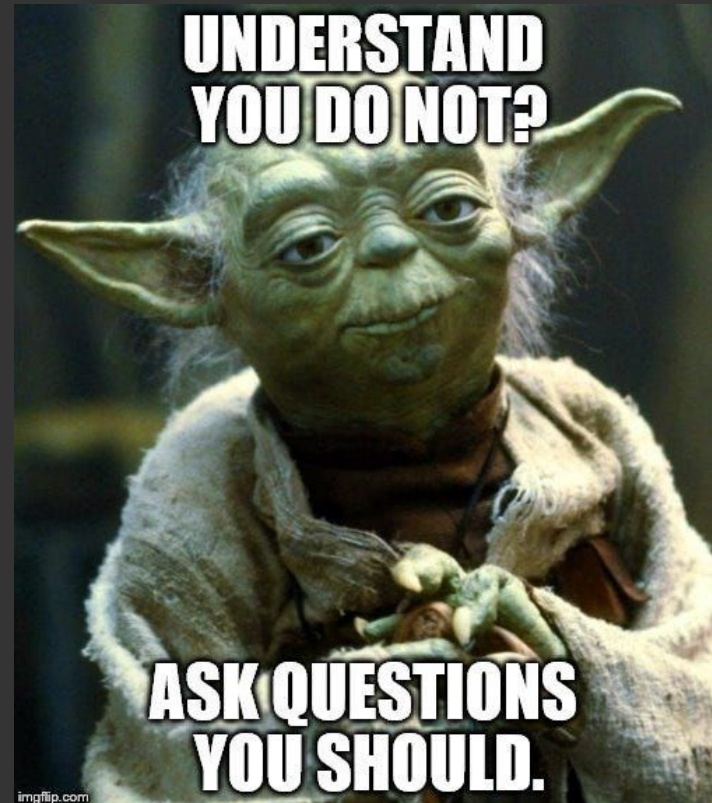




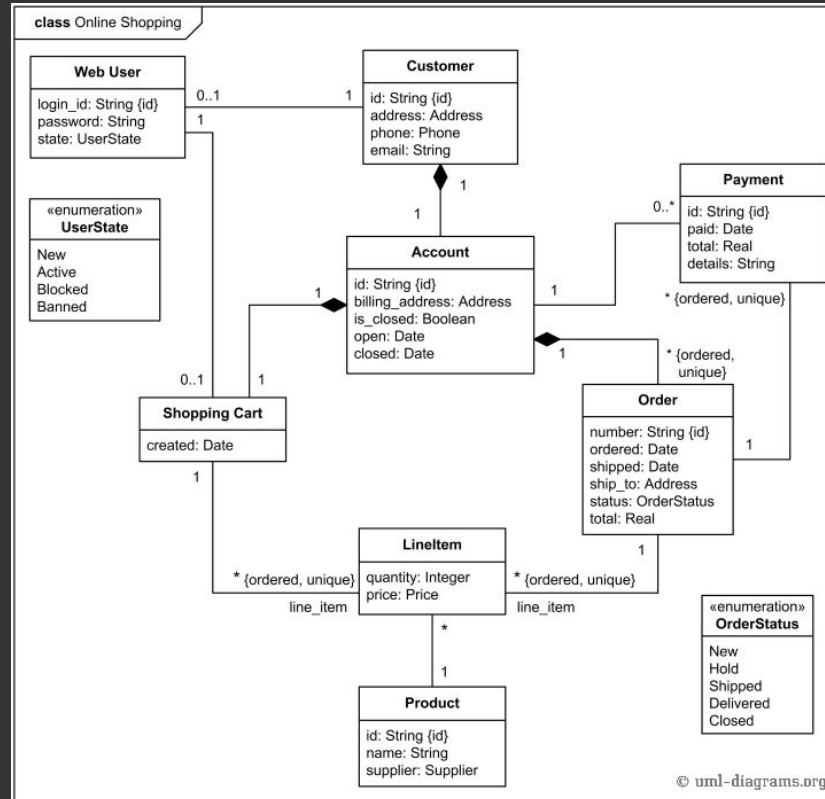
makeameme.org

Follow up Questions

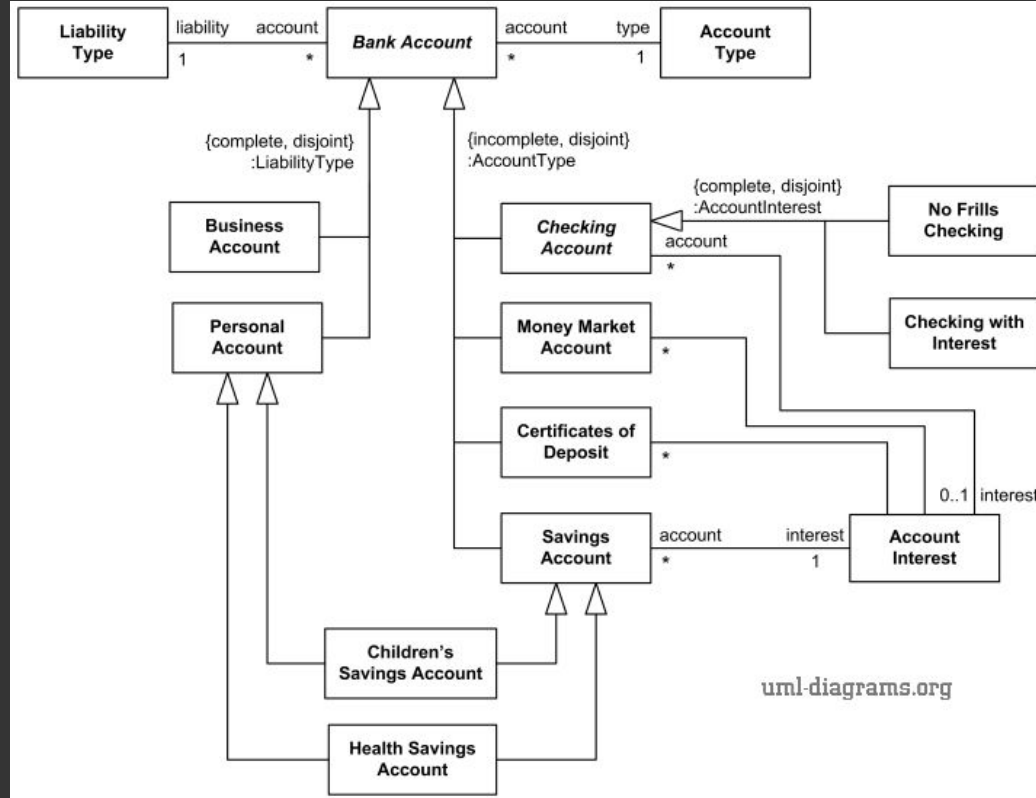
- Which relationships are composition vs aggregation and why?
- Would it make sense for Loan to hold a reference to Librarian?
- Could Member implement a User interface for future Admin types?
- What if a book has multiple authors?
- How would you support DVD loans with minimal disruption to the existing design?



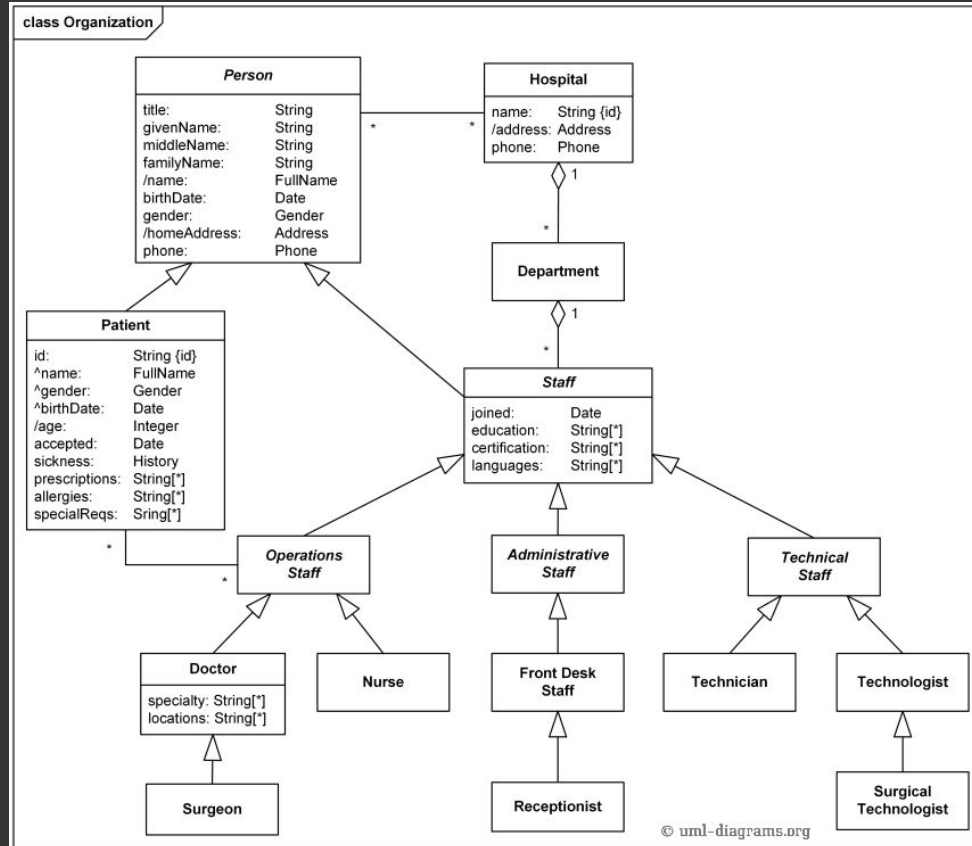
Shopping System



Bank Account



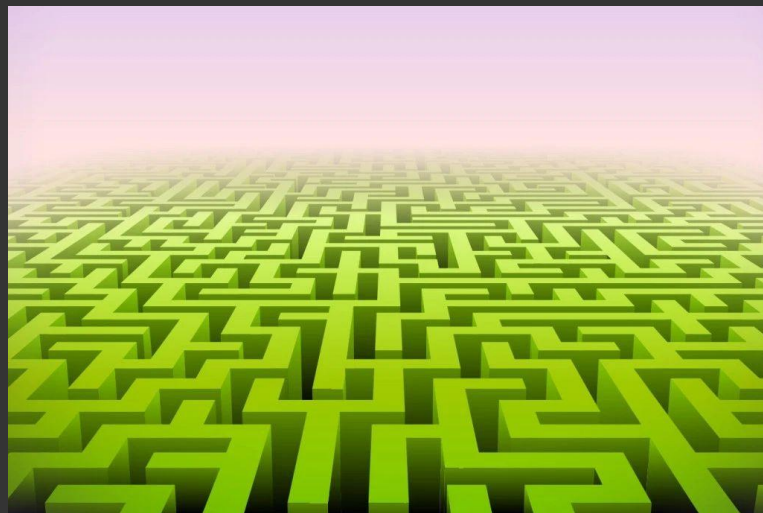
Hospital



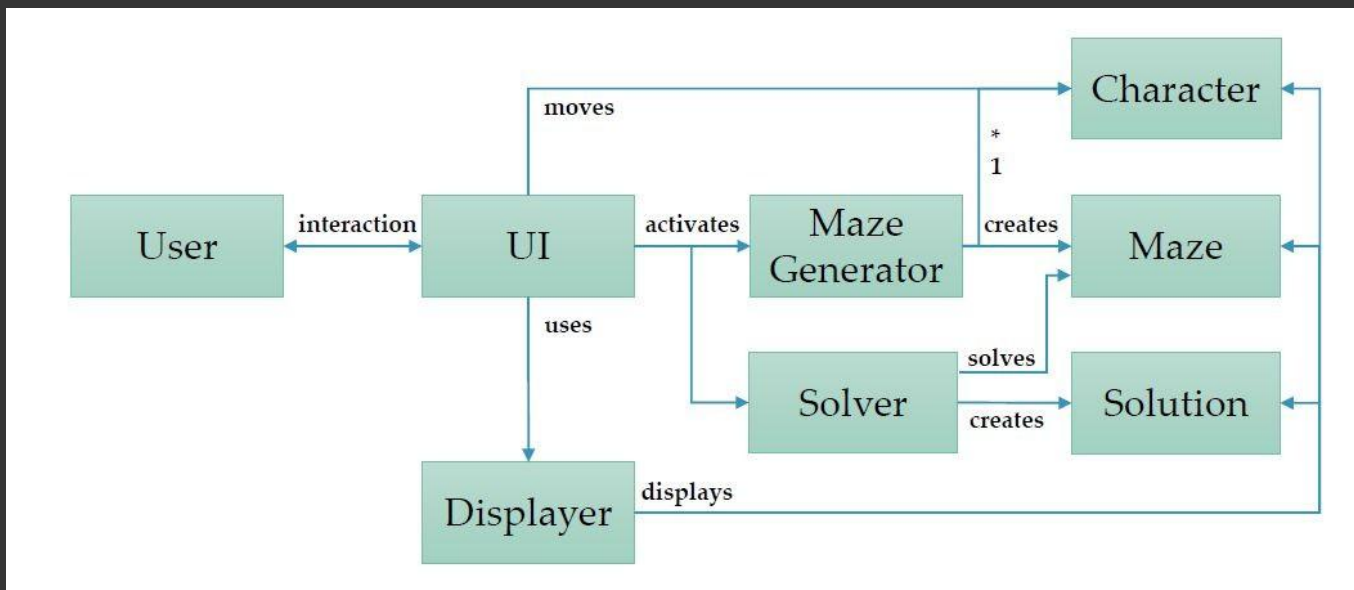
The Maze Example

We would like to create a maze game

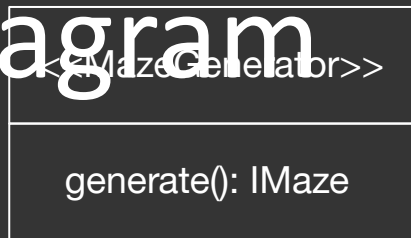
1. User interface
2. Display the maze
3. Create maze
4. Solve the maze
5. Move the character



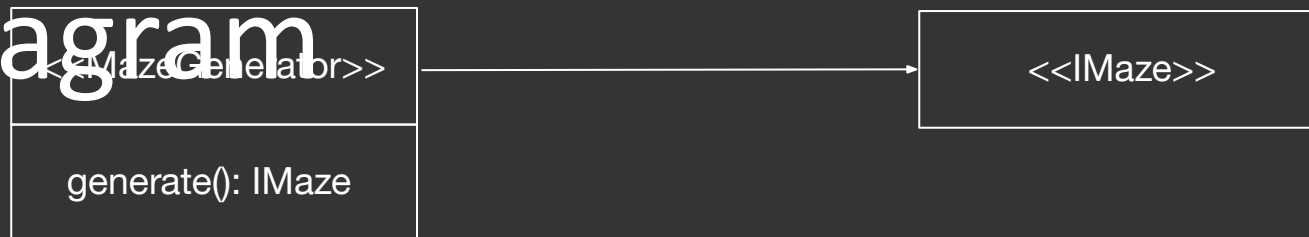
Conceptual model



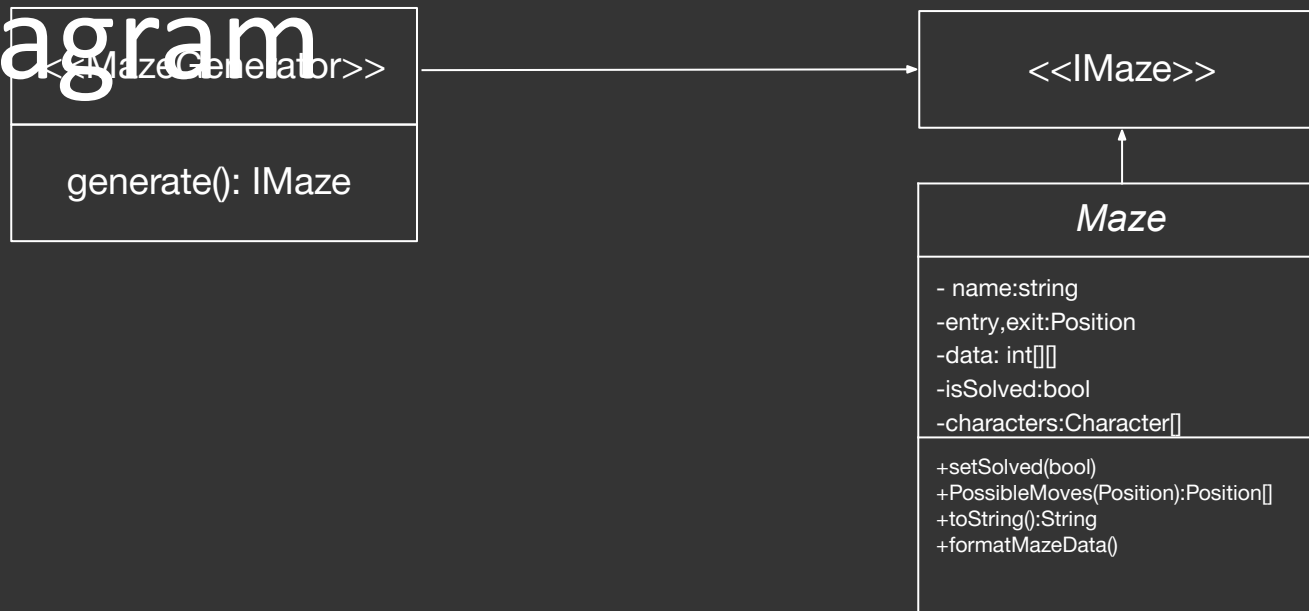
Class diagram



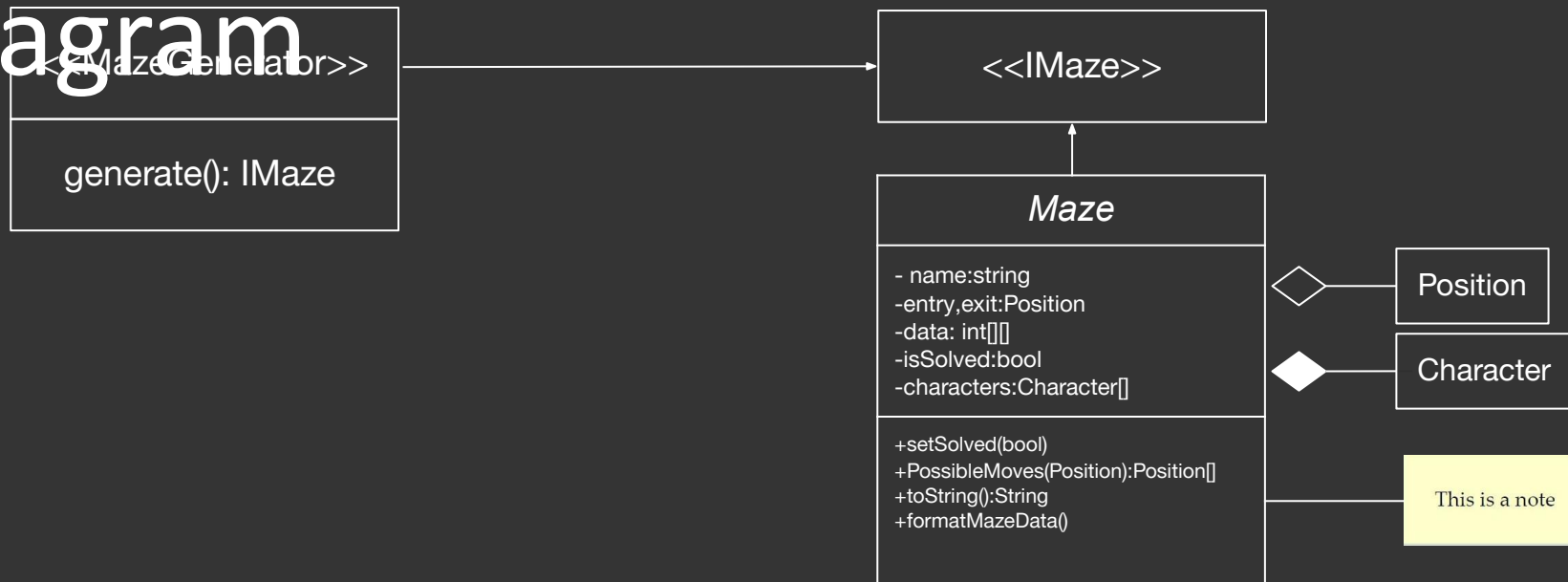
Class diagram



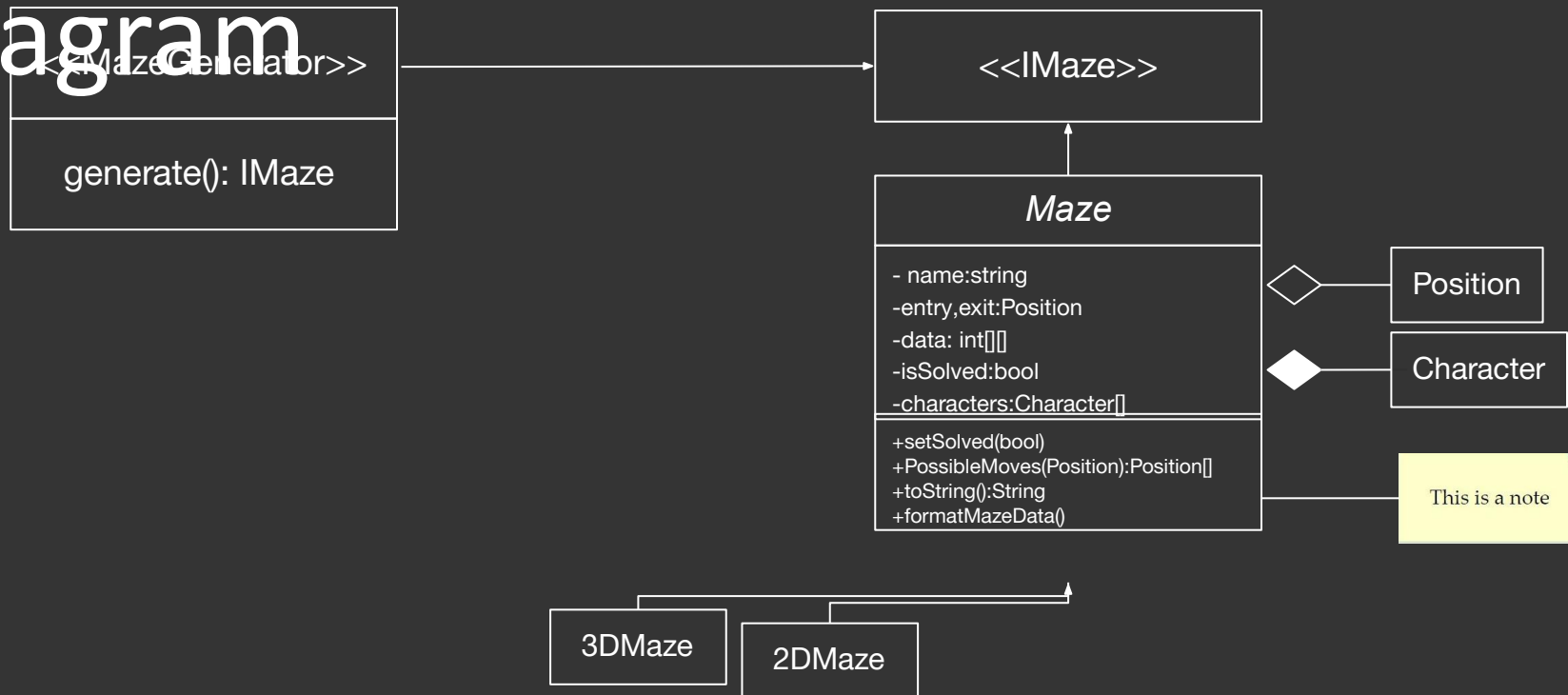
Class diagram



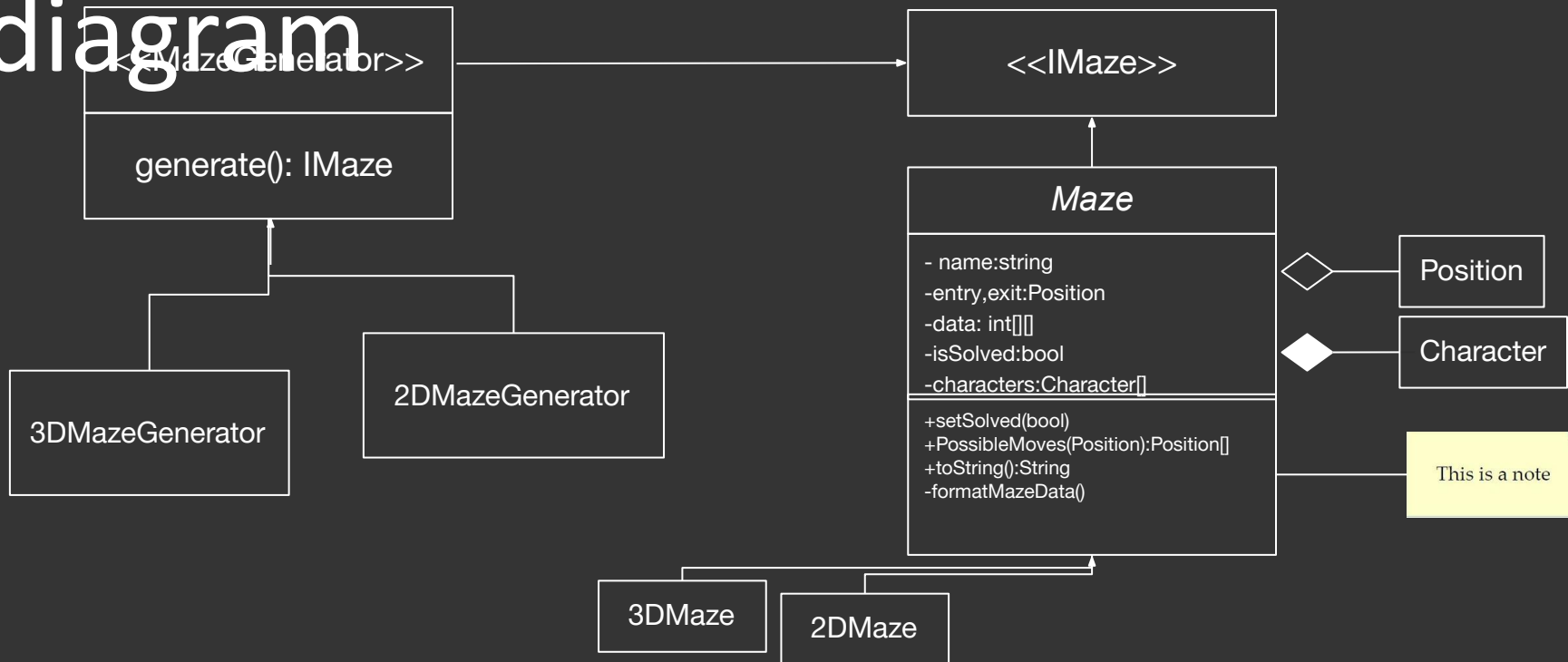
Class diagram



Class diagram



Class diagram



Class diagram

